

Reson8 Platform — Phase 1

Product Requirements Document

Feature Specifications · All 76 Phase 1 Features

Prepared by: ID8 Media · Product Engineering

Date: June 2026

Version: 1.0

Team: Tech Engineer · Product Engineer

Document Structure

Each feature is defined with: - **Description** — What it does and why it matters - **User Story** — Who uses it, what they need, why - **Acceptance Criteria** — Specific, testable conditions for “done” - **Owner** — Tech Engineer (backend) · Product Engineer (UI/config) · Both - **Sprint** — Delivery sprint - **Dependencies** — Features that must be complete first

Roles & Glossary

Term	Definition
Producer	Selects and stages graphics for editorial review
Editor	Reviews, approves, or rejects staged graphics
Operator	Takes editor-approved graphics to air via Vizrt
Senior Editor	Has emergency override authority
Admin	Configures system: elections, users, feeds, templates
Editorial Queue	Holding area for graphics awaiting editor review

Term	Definition
Take to Air	Action that sends a graphic to the Vizrt renderer for broadcast
Vizrt	The graphics rendering system connected to the broadcast output
Sentra Engine	The AI layer handling NLP, sentiment, classification, content generation
FAL	Feed Abstraction Layer — generic connector for all external data feeds
SLA	Service Level Agreement — ≤4 seconds from feed event to graphic on screen
RTL	Right-to-left text layout, required for Arabic

Section 1 — Sentra Engine (Sprint 1)

The Engine is always active. These capabilities are infrastructure — they cannot be toggled off and are shared by every module.

E1 — 10-Language NLP

Sprint: S1 · **Owner:** Tech Engineer

Description:

The Engine reads, understands, and processes content in all 10 approved languages: English, Arabic, French, Spanish, Russian, Chinese, German, Portuguese, Persian, and Turkish. Arabic dialect recognition (Egyptian, Gulf, Levantine, Maghrebi) is included alongside Modern Standard Arabic. All downstream analysis — sentiment, entity extraction, classification — operates natively in each language without translation.

User Story:

As the platform, I need to process incoming content in any of the 10 approved languages so that broadcasters monitoring multilingual news environments receive accurate analysis regardless of the source language.

Acceptance Criteria: - [] Content in each of the 10 languages is parsed and processed without language-detection errors - [] Arabic dialect content is correctly classified (MSA vs. Egyptian vs. Gulf vs. Levantine vs. Maghrebi) -

[] Downstream features (sentiment, NER, breaking detection) produce results for all 10 languages - [] No language falls back to machine translation before analysis

Dependencies: None (foundational)

E2 — Sentiment Analysis

Sprint: S1 · **Owner:** Tech Engineer

Description:

Assigns a Positive, Neutral, or Negative sentiment score to every piece of ingested content, with a confidence percentage. Sentiment is language-aware and culture-adjusted — a phrase that is positive in one language is not incorrectly scored as negative due to direct translation assumptions. Scores are produced per mention and can be aggregated per topic, entity, or time window.

User Story:

As an Editor reviewing candidate coverage, I want to see the live sentiment of public mentions for each candidate so that I can contextualise results data with how the public is reacting online.

Acceptance Criteria: - [] Every ingested social post and news article receives a sentiment label (Positive / Neutral / Negative) and a confidence score (0–100%) - [] Sentiment scoring works correctly in all 10 languages without cross-language drift - [] Aggregated sentiment per entity (candidate, party, team) updates in real time as new content arrives - [] Culture-specific idioms in Arabic do not produce inverted scores - [] Confidence score of <40% routes item as “Uncertain” rather than forcing a label

Dependencies: E1

E3 — 6-Emotion Model

Sprint: S1 · **Owner:** Tech Engineer

Description:

Beyond positive/negative, classifies content into one of six emotions: Anger, Fear, Joy, Sadness, Disgust, or Surprise. Applied per mention and per topic, the emotion model reveals the quality of public reaction — not just its direction. A winning election result may show high Joy among supporters and high Fear among the opposition simultaneously.

User Story:

As a Producer preparing election night graphics, I want to understand not just whether sentiment is positive or negative but what emotion is dominant so that the editorial team can frame graphics with accurate context.

Acceptance Criteria: - [] Every processed content item receives one primary emotion label from the six-class model - [] Emotion distribution across a set of mentions can be queried per entity or topic - [] Emotion output is available in the same data feed as sentiment scores - [] Model produces valid output in all 10 languages

Dependencies: E1, E2

E4 — Named Entity Recognition (NER)

Sprint: S1 · **Owner:** Tech Engineer

Description:

Automatically identifies and classifies named entities in all ingested content: Persons (candidates, politicians, players, executives), Organisations (parties, clubs, companies, government bodies), Locations (countries, cities, constituencies, stadiums), and Events (elections, matches, market events). Extracted entities are linked to platform records where a match exists, enabling real-time association of mentions to known candidates, parties, or constituencies.

User Story:

As the platform, I need to know which candidate or party a social post is about so that sentiment and emotion scores can be attributed to the correct entity rather than stored as unlinked data.

Acceptance Criteria: - [] Person, Organisation, Location, and Event entities are extracted from all 10 languages - [] Extracted entities are matched to platform records (candidates, parties, constituencies) when a match exists with >85% confidence - [] Unmatched entities are surfaced as new entity candidates for admin review - [] Cross-language entity resolution works: “Macron” in English and “ماكرون” in Arabic link to the same entity record

Dependencies: E1

E5 — AI Content Generation

Sprint: S1 · **Owner:** Tech Engineer

Description:

Generates broadcast-ready text from live event data: headlines, lower-third captions, and talking points. Output is always bilingual (Arabic + English). All generated content is routed to the editorial queue — it never goes to air without editor review. Generation is event-triggered: a new election result, a goal, or a market threshold crossing fires the generation pipeline automatically.

User Story:

As an Editor, I want a pre-written headline and caption to appear in my queue the moment a significant event is detected so that I can approve and take it to air faster than writing copy from scratch.

Acceptance Criteria: - [] Headline generated within the ≤ 4 second SLA from event detection - [] All generated content is in Arabic and English simultaneously - [] Generated content never reaches Vizrt without passing through C7 (Editorial Review Gate) - [] Character limits per graphic type (headline, lower-third) are respected in generation output - [] Editors can edit generated copy before approval

Dependencies: E1, E4, C7

E6 — Real-Time Pipeline

Sprint: S1 · **Owner:** Tech Engineer

Description:

The continuous processing backbone: ingests 10,000+ signals per day from all connected sources, enriches each signal with sentiment, emotion, and entity data, and embeds the result into the platform's knowledge layer. Runs hourly topic clustering and deduplication to prevent the same story from appearing multiple times across sources. All downstream features draw from this processed, enriched signal stream.

User Story:

As the platform, I need to continuously ingest and process thousands of signals per day without manual intervention so that editors always see enriched, deduplicated content — not raw data.

Acceptance Criteria: - [] Pipeline processes a minimum of 10,000 signals per 24-hour period without degradation - [] End-to-end latency from signal ingestion to enriched output is ≤ 30 seconds under normal load - [] Deduplication correctly suppresses re-runs of the same story across different sources (same story detected in >3 outlets is clustered, not listed)

separately) - [] Pipeline health is visible in the Control Room dashboard - [] Pipeline continues operating during a single component failure (resilient design)

Dependencies: E1, E2, E3, E4

E7 — Breaking News Signal

Sprint: S1 · **Owner:** Tech Engineer

Description:

Classifies incoming events as Breaking, Developing, or Routine in real time. Severity scoring is based on: source credibility, rate of spread, cross-source confirmation count, and entity significance. A single highly-credible source reporting an unprecedented event can trigger Breaking classification. The signal feeds the FLASH Breaking News module and the breaking sub-feeds in ORA and ARENA.

User Story:

As an Editor, I want the system to automatically surface events it classifies as Breaking or Major Breaking so that I am never the last to know about a significant story.

Acceptance Criteria: - [] Events classified as Major Breaking, Breaking, or Developing appear in the editorial alert queue within 10 seconds of classification - [] Classification uses a minimum of: source credibility score, velocity, and cross-source confirmation - [] False positive rate (Routine events classified as Breaking) is <5% in testing - [] Breaking signal fires across all 10 monitored languages simultaneously

Dependencies: E1, E6, E12

E8 — Anomaly Detection

Sprint: S1 · **Owner:** Tech Engineer

Description:

Monitors volume and velocity of content signals and alerts when a statistically unusual spike occurs — even before that spike reaches the Breaking classification threshold. Used to surface early warning signals: a sudden 10× increase in mentions of a candidate, a sharp drop in a stock's mention volume, or an unusual pattern in results data. Separate from but complementary to E7.

User Story:

As an Editor, I want to be alerted to abnormal spikes in activity around any topic or entity so that I can investigate a story before it reaches breaking news threshold.

Acceptance Criteria: - [] Anomaly alerts fire when a measured signal deviates more than 3 standard deviations from the rolling 24-hour baseline - [] Alert includes: entity/topic affected, current volume, baseline volume, deviation magnitude - [] Anomaly alerts surface in the editorial queue separately from breaking news items - [] System does not alert on normal election-night spikes that are expected (baseline recalibration supported)

Dependencies: E6

E9 — Arabic Processing

Sprint: S1 · **Owner:** Tech Engineer

Description:

Full Arabic language support at production broadcast quality. Includes right-to-left text rendering, Shams typeface integration for broadcast display, Modern Standard Arabic and regional dialect processing (Egyptian, Gulf, Levantine, Maghrebi), and bidirectional text handling for mixed Arabic/English content. All Arabic output meets broadcast typography standards.

User Story:

As a broadcaster producing Arabic-language output, I need all text — on-screen graphics, captions, and system interfaces — to render correctly in Arabic so that there are no typography or layout errors on air.

Acceptance Criteria: - [] All Arabic text renders right-to-left in all UI and Vizrt output surfaces - [] Shams typeface is applied to all Arabic broadcast text output - [] Mixed Arabic/English text (bidirectional) displays correctly without character order reversal - [] Dialect detection correctly classifies content as MSA, Egyptian, Gulf, Levantine, or Maghrebi - [] Arabic character diacritics (tashkeel) are preserved and displayed correctly

Dependencies: E1

E10 — Multi-Model AI Routing

Sprint: S1 · **Owner:** Tech Engineer

Description:

The platform routes AI tasks to the optimal large language model based on task type: Gemini 2.5 Flash for high-speed, high-volume classification tasks; GPT-4 for nuanced content generation; Grok for real-time social analysis. Model selection is configured by the Admin and can be switched at runtime without restarting the platform. All models share the same API wrapper — switching a model requires only a configuration change.

User Story:

As an Admin, I want to be able to switch the AI model powering specific tasks without redeploying the platform so that I can optimise for cost, speed, or quality as needed.

Acceptance Criteria: - [] Admin can select the active model per task category (classification, generation, analysis) from the admin panel - [] Model switch takes effect within 60 seconds without platform restart - [] All three models (Gemini 2.5 Flash, GPT-4, Grok) are integrated and callable - [] Model fallback: if the primary model is unavailable, traffic automatically routes to the secondary model - [] Model selection does not affect output format or downstream feature behaviour

Dependencies: None (foundational)

E11 — Velocity & Virality Detection

Sprint: S1 · **Owner:** Tech Engineer

Description:

Tracks the rate-of-change of content signal volume in real time. Detects when a topic or entity is accelerating toward viral status — before it peaks. The early viral signal allows editorial teams to prepare graphics and coverage before the story saturates the feed. Distinguishes between natural viral growth and coordinated amplification (sudden spikes with low source diversity).

User Story:

As an Editor, I want advance warning when a topic is going viral so that I have time to prepare coverage before it becomes the dominant story in the feed.

Acceptance Criteria: - [] Virality signal fires when a topic's mention rate doubles in a 15-minute window - [] Signal includes: topic/entity, current velocity, projected peak, source diversity score - [] Low source diversity (coordinated amplification) is flagged separately from organic viral growth - [] Virality signals surface in the editorial queue with a distinct "Trending" label

Dependencies: E6, E8

E12 — Source Credibility Scoring

Sprint: S1 · **Owner:** Tech Engineer

Description:

Assigns a trust weight to every ingested content source. Authoritative sources (official government feeds, wire services, verified broadcaster accounts) receive a 1.3× credibility multiplier. Organisational uploads (press releases, official club communications) receive 1.2×. Unverified social accounts receive a baseline score of 1.0×. Credibility weights are applied in breaking news classification, anomaly detection, and content surfacing — high-credibility signals are prioritised in the editorial queue.

User Story:

As an Editor, I want content from credible sources to surface above unverified social posts so that I am reviewing the most reliable information first.

Acceptance Criteria: - [] Every source in the system has an assigned credibility tier (Authoritative / Organisational / Standard) - [] Credibility multipliers (1.3×, 1.2×, 1.0×) are applied to ranking and breaking classification scores - [] Admin can add new sources and assign them a credibility tier - [] Breaking news classification from a single authoritative source outweighs the same content from 5 unverified sources - [] Credibility score is visible alongside each item in the editorial queue

Dependencies: E6, E7

Section 2 — Reson8 Core: Editorial Chain (Sprint 2)

C1 — Producer Preparation Queue

Sprint: S2 · **Owner:** Both (Tech: backend state machine · Product: queue UI)

Description:

The Producer's primary workspace. Displays all available graphics for the current broadcast context, populated with live data from the active module (election results, match data, market prices). The Producer selects a graphic

type, reviews the auto-populated data, makes any manual edits, and stages it for editorial review. The graphic enters the editorial queue only when the Producer explicitly submits it.

User Story:

As a Producer, I want a single screen showing all available graphics with live data already filled in so that I can select, review, and stage a graphic for air in seconds rather than manually entering data.

Acceptance Criteria: - [] Producer sees a list of all available graphic types for the active module - [] Each graphic type shows a live preview populated with current data - [] Producer can edit any data field before staging - [] “Stage for Review” action sends the graphic to the editorial queue with a status of “Pending” - [] Producer can stage multiple graphics simultaneously; each appears as a separate queue item - [] Staged graphics show the Producer’s name and timestamp in the audit trail

Dependencies: E5, C5, C6

C2 — Editor Approval Layer

Sprint: S2 · **Owner:** Both (Tech: approval logic · Product: approval UI)

Description:

The Editor’s primary decision screen. Displays every staged graphic with a full preview, the data that will populate it, and the AI-generated copy (headline/caption) if applicable. The Editor can approve (sending it to the operator queue), reject with a note (returning it to the Producer), or hold for further review. Every decision is timestamped and logged immutably. Editors can annotate graphics with inline comments visible to the Producer.

User Story:

As an Editor, I want to see a full preview of every staged graphic with all copy and data clearly displayed so that I can make an approval decision quickly and confidently without asking the Producer for clarification.

Acceptance Criteria: - [] Editor sees: graphic preview, all data fields, AI-generated copy (if applicable), Producer name, staging timestamp - [] Approve action moves the graphic to the Operator queue with status “Approved” - [] Reject action requires a mandatory rejection note and returns the graphic to the Producer with status “Returned” - [] Hold action keeps the graphic in the Editor queue with status “On Hold” — no time limit - [] Editor cannot take a graphic to air directly; only Operator can (role separation enforced) - [] All Editor decisions are logged in the Editorial Audit Trail (C6) with timestamp and username

Dependencies: C1, C5, C6, C7

C3 — Operator Take to Air

Sprint: S2 · **Owner:** Both (Tech: Vizrt trigger · Product: operator UI)

Description:

The Operator's action screen. Displays all editor-approved graphics ready for air, with the graphic type, content, and the editor who approved it. The Operator selects a graphic and fires it to the Vizrt renderer with a single action. The moment the take-to-air action executes, an immutable log entry is created (operator, graphic, timestamp, Vizrt output target). A confirmation receipt appears on screen.

User Story:

As an Operator, I want to fire an approved graphic to the Vizrt renderer with one click so that the graphic reaches air within the ≤4 second SLA without any manual data entry.

Acceptance Criteria: - [] Operator sees only editor-approved graphics in their queue - [] Take-to-air fires to the correct Vizrt renderer output within 1 second of button press - [] Audit log entry is created within 500ms of the take-to-air action - [] Confirmation receipt appears on screen confirming the graphic is on air - [] Operator cannot approve a graphic; only Editors can (role separation enforced) - [] If Vizrt renderer is unreachable, Operator receives an explicit error — the action does not silently fail

Dependencies: C2, C6, C8, C9, C10

C4 — Reject & Revise Workflow

Sprint: S2 · **Owner:** Both (Tech: revision state machine · Product: revision UI)

Description:

When an Editor rejects a graphic, it re-enters the Producer's queue with a "Returned" status and the Editor's rejection note visible. The Producer can revise the content and re-submit. The cycle repeats until the Editor approves or the graphic is discarded. Each revision cycle is tracked, and the full revision history (original + all revisions + rejection notes) is preserved in the audit trail.

User Story:

As a Producer, I want to receive a clear rejection note when an editor returns a graphic so that I know exactly what to fix and can resubmit quickly.

Acceptance Criteria: - [] Returned graphic appears at the top of the Producer's queue with "Returned" status and the Editor's note prominently displayed - [] Producer can edit and resubmit; the graphic re-enters the Editor's queue with "Revised" status - [] Full revision history (each version + rejection note) is stored and visible in the audit trail - [] No limit on revision cycles; the workflow continues until approved or discarded - [] Producer can discard a returned graphic (removes it from all queues with a "Discarded" audit log entry)

Dependencies: C1, C2, C6

C5 — Live / Preview Separation

Sprint: S2 · **Owner:** Product Engineer

Description:

Every graphic is previewed in a dedicated preview pane before it is staged or taken to air. The preview renders the actual Vizrt scene with live data — what the Producer and Editor see in the preview is exactly what will appear on screen. There is a clear visual distinction between "Preview" and "On Air" states. No accidental on-air action is possible from the preview pane.

User Story:

As a Producer, I want to see exactly how a graphic will look on screen before staging it so that I can catch visual or data errors before they reach the editorial queue.

Acceptance Criteria: - [] Preview pane renders the actual Vizrt scene (not a mock-up) with live current data - [] Preview and "On Air" states are visually distinct (colour coding, label, border) - [] Preview updates in real time as underlying data changes - [] No take-to-air action is available from the preview pane — it is read-only - [] Preview latency from data change to visual update is ≤ 2 seconds

Dependencies: C8, C9

C6 — Editorial Audit Trail

Sprint: S2 · **Owner:** Tech Engineer

Description:

An immutable, tamper-proof log of every action taken on the platform. Records: who (user + role), what (action type), when (UTC timestamp),

millisecond precision), and what changed (graphic ID, content snapshot). The audit trail cannot be edited, deleted, or filtered by any user, including Admin. It is the legal and editorial record of everything that went to air.

User Story:

As a Broadcaster, I need a complete, tamper-proof record of every editorial decision and on-air action so that I can demonstrate accountability and investigate any disputed broadcast.

Acceptance Criteria: - [] Every platform action (stage, approve, reject, take to air, override, login, logout) creates an audit entry - [] Each entry contains: user ID, role, action type, timestamp (UTC, millisecond), entity affected, content snapshot - [] No user — including Admin — can edit or delete an audit entry - [] Audit log is exportable to CSV/PDF by Admin - [] Audit log search supports filtering by: user, action type, date range, module, graphic type - [] Log write fails safe: if the audit log is unavailable, the action that would have been logged is blocked (not silently skipped)

Dependencies: None (foundational — all other features depend on this)

C7 — Editorial Review Gate

Sprint: S2 · **Owner:** Tech Engineer

Description:

A mandatory enforcement layer that blocks all AI-generated content from reaching Vizrt without explicit Editor approval. Applied automatically to any graphic whose headline, caption, or body copy was generated by E5. The gate is not bypassable by any user role, including Admin. It ensures the human-in-the-loop principle is enforced at the system level, not relying on process alone.

User Story:

As a Broadcaster, I need a technical guarantee that no AI-generated text ever reaches air without a human editor reviewing and approving it so that I am never exposed to automated broadcast errors.

Acceptance Criteria: - [] Any graphic containing AI-generated text (from E5) is automatically flagged in the editorial queue with a “Requires Editorial Review” indicator - [] The take-to-air action (C3) is disabled for flagged graphics until an Editor explicitly approves - [] No role — including Admin or Senior Editor — can bypass the gate; they must approve, not override it - [] The gate applies to all AI-generated text fields: headline, lower-third caption, ticker copy, talking points - [] A graphic with manually edited (non-AI) copy is not blocked by the gate

Dependencies: C2, C6, E5

Section 3 — Reson8 Core: Vizrt Integration (Sprint 2)

C8 — Vizrt Scene Binding

Sprint: S2 · **Owner:** Tech Engineer

Description:

Maps every platform data field to its corresponding Vizrt scene data container. A “scene binding” defines: which Vizrt scene is used for a given graphic type, which platform data field populates each data container in the scene, and what formatting rules apply (font, RTL/LTR, field length). Bindings are defined once per graphic type and reused on every take-to-air. Admin can update bindings without code changes.

User Story:

As a Tech Engineer, I need to configure how platform data maps to Vizrt scene fields once so that every take-to-air for a given graphic type automatically sends the correct data to the correct Vizrt container without manual mapping.

Acceptance Criteria: - [] Each graphic type has a saved binding: scene ID + data container ID + platform data field + formatting rules - [] Admin can add, edit, or remove bindings via the admin panel without code deployment - [] Binding validation: if a data field referenced in a binding does not exist in the current data context, the binding fails gracefully with a clear error (not a silent null) - [] Multi-scene support: one graphic type can bind to multiple Vizrt scenes (e.g., a results graphic that updates both the results bar and the map)

Dependencies: None (foundational for all Vizrt features)

C9 — Direct Data Feed to Vizrt

Sprint: S2 · **Owner:** Tech Engineer

Description:

Pushes live data to the Vizrt renderer in real time, replacing the need for any manual data entry at the Vizrt console. When a take-to-air is executed, the platform sends all bound data fields directly to the Vizrt engine. If the

underlying data changes while a graphic is on air (e.g., a vote count updates), the platform can push the update to the live scene without requiring a new take-to-air action.

User Story:

As an Operator, I want vote count data to appear on the Vizrt graphic automatically without me typing numbers into a Vizrt console so that I can focus on timing takes rather than data entry.

Acceptance Criteria: - ☐ Take-to-air sends all bound data fields to Vizrt within 500ms - ☐ Live data updates to an on-air scene (e.g., vote count increment) are pushed to Vizrt within the SLA - ☐ No manual data entry at the Vizrt console is required for any platform-driven graphic - ☐ Data push is atomic: either all fields update or none do (partial updates are not permitted)

Dependencies: C3, C8

C10 — Viz Engine Output

Sprint: S2 · **Owner:** Tech Engineer

Description:

Connects to the broadcaster's existing Vizrt renderer (Viz Engine). The platform drives the existing renderer — it does not replace the graphics engine or require a new one. The connection uses the Viz Engine's native data protocol. The platform supports multiple concurrent Viz Engine connections (for multi-output setups).

User Story:

As a Broadcaster, I want the platform to connect to my existing Vizrt infrastructure so that I do not need to purchase, configure, or learn a new rendering system.

Acceptance Criteria: - ☐ Platform connects to Viz Engine using the native Viz Engine data protocol (TCP/Viz command set) - ☐ Connection established without modifying the broadcaster's existing Viz Engine installation - ☐ Platform supports minimum 2 concurrent Viz Engine connections in Phase 1 - ☐ Connection health is visible in the Control Room status monitor (C14) - ☐ Connection failure triggers an alert and automatic reconnection attempt

Dependencies: C8

Section 4 — Reson8 Core: Access & Security (Sprint 3)

C20 — Role-Based Access Control (RBAC)

Sprint: S3 · **Owner:** Tech Engineer

Description:

Four distinct roles with non-overlapping permissions: Producer (can stage, can view, cannot approve or take to air), Editor (can approve/reject, cannot take to air), Operator (can take to air approved graphics only, cannot approve), Admin (can configure all system settings, cannot take editorial actions). Role boundaries are enforced at the API layer — the UI reflects what is permitted, but the enforcement is at the data and function level.

User Story:

As a Broadcaster, I need role boundaries to be technically enforced so that a Producer cannot accidentally (or deliberately) take a graphic to air without editorial approval.

Acceptance Criteria: - [] Producer API: stage, view queue, view preview — approve, take to air, and config endpoints return 403 - [] Editor API: approve, reject, annotate — take to air and config endpoints return 403 - [] Operator API: take-to-air for approved items only — approve, stage, and config endpoints return 403 - [] Admin API: all configuration endpoints — editorial action endpoints return 403 - [] Role assignment is per-tenant and per-module where applicable - [] Role changes take effect immediately without requiring re-login

Dependencies: C24

C21 — Audit Logging

Sprint: S3 · **Owner:** Tech Engineer

Description:

Platform-wide immutable event log covering all system events beyond the editorial chain: user logins/logouts, configuration changes, role assignments, feed connection events, failover events, and system errors. Complements C6 (Editorial Audit Trail) which covers editorial actions specifically. Together, C6 and C21 form a complete audit record of every action on the platform.

User Story:

As an Admin, I want a complete log of all system events — not just editorial actions — so that I can investigate configuration changes, login anomalies, or feed failures after the fact.

Acceptance Criteria: - [] Log captures: login, logout, failed login attempt, role assignment change, config change, feed connect/disconnect, failover event, system error - [] Each log entry includes: timestamp (UTC, ms), user, action, parameters changed (before/after for config changes) - [] Log is separate from but linked to C6 (editorial audit trail) by shared event IDs - [] Admin can search and filter the system audit log - [] Log is tamper-proof: no delete, no edit for any role - [] Failed login attempts exceeding 5 in 10 minutes trigger an alert to Admin

Dependencies: C20, C24

C22 — Multi-Tenant Architecture

Sprint: S3 · **Owner:** Tech Engineer

Description:

Each broadcaster client is a separate, fully isolated tenant. Tenant data (users, content, configuration, queues, audit logs) is partitioned at the database level — no tenant can access another's data, even with a valid session token. The platform can host multiple clients simultaneously on the same infrastructure without data leakage risk.

User Story:

As a client broadcaster, I need a guarantee that my editorial data, configuration, and audit logs are inaccessible to other clients on the same platform so that my broadcast operations remain confidential.

Acceptance Criteria: - [] All data queries are scoped to the authenticated tenant's identifier — cross-tenant queries are architecturally impossible, not just unenforced - [] Tenant isolation verified by penetration test: authenticated user of Tenant A cannot access Tenant B resources via any API endpoint - [] Tenant onboarding creates a fully isolated data partition — no shared tables, no shared queues - [] Tenant offboarding cleanly removes all data from all partitions - [] Admin accounts are per-tenant — there is no super-admin that crosses tenant boundaries in production

Dependencies: C20, C24

C23 — Notification System

Sprint: S3 · **Owner:** Product Engineer

Description:

Delivers in-app and push notifications to relevant users when platform events require attention: a new graphic enters the Editor's queue, a graphic is returned to the Producer, a feed goes down, a threshold alert fires, or an emergency override is executed. Notifications are configurable per user — each user sets which event types they want to be notified about and by which channel (in-app, push).

User Story:

As an Editor, I want to receive an instant notification when a new graphic is staged for my review so that I do not need to watch the queue screen continuously.

Acceptance Criteria: - [] In-app notifications appear within 3 seconds of the triggering event - [] Push notification is delivered within 10 seconds of the triggering event - [] Each user can configure which event types trigger notifications (per event category) - [] Each user can configure delivery channel per event type (in-app only, push only, both) - [] Notification includes: event type, graphic type, triggered by (user), timestamp — sufficient to act without opening the full screen

Dependencies: C20

C24 — Broadcast Security & Auth

Sprint: S3 · **Owner:** Tech Engineer

Description:

Authentication layer for the broadcast control room environment. Supports: JWT session tokens (standard login), OTP (one-time password for shared terminals), 2FA (two-factor authentication for all editorial roles), SSO (single sign-on for enterprise broadcast environments), and magic link (email-based passwordless login). Session management is hardened for broadcast: sessions do not expire mid-broadcast (sessions are time-extended when active) but expire after 30 minutes of inactivity.

User Story:

As a Broadcaster, I need secure authentication that works in a fast-paced control room environment — including shared terminals — without forcing staff to re-authenticate at critical broadcast moments.

Acceptance Criteria: - [] All five auth methods (JWT, OTP, 2FA, SSO, magic link) are implemented and selectable per tenant - [] Active sessions extend automatically when the user is active (no mid-broadcast expiry) - [] Sessions expire after 30 minutes of inactivity - [] 2FA is enforced for all Editor and Senior Editor roles (cannot be disabled) - [] OTP generates a new valid code every 60 seconds - [] SSO integrates via SAML 2.0 or OAuth 2.0 - [] All auth events are logged in C21 (Audit Logging)

Dependencies: None (foundational security layer)

Section 5 — Reson8 Core: Control Room (Sprints 3–4)

C11 — Control Room Dashboard

Sprint: S3 · **Owner:** Both (Tech: aggregation API · Product: dashboard UI)

Description:

A single-screen overview of the entire broadcast operation. Shows: all active Vizrt outputs and their current on-air graphic, the editorial queue with status indicators, queue depth per module, next-up graphic preview, feed health indicators, and active user presence per role. The dashboard is the Operator's and Senior Editor's command centre during a broadcast. It does not replace the individual module views — it aggregates them.

User Story:

As an Operator, I want a single screen showing everything that is happening across all outputs and all queues so that I can manage a live broadcast without switching between multiple screens.

Acceptance Criteria: - [] Dashboard shows: all Vizrt output states (on-air graphic per output), editorial queue depth per module, next-up approved graphic per output, feed health status, active users (by role) - [] All data refreshes without page reload (real-time via WebSocket) - [] Dashboard is viewable on screens from 1920×1080 to 3840×2160 without scrolling for core information - [] Dashboard load time from login: ≤3 seconds - [] Dashboard data is tenant-scoped (no cross-tenant data visible)

Dependencies: C14, C20, C10

C12 — Multi-Screen Management

Sprint: S3 · **Owner:** Both (Tech: output routing · Product: screen assignment UI)

Description:

Assigns specific Vizrt outputs (physical screens or channels) to specific graphic queues. For a broadcaster running simultaneous election coverage on three channels, each channel has its own output queue. An Operator can be assigned responsibility for one or more specific outputs. Graphics staged for Channel 1 only appear in Channel 1's queue and only fire to Channel 1's Vizrt renderer.

User Story:

As an Operator managing two simultaneous channel outputs, I want to assign graphics to specific channels so that a graphic prepared for my English channel does not accidentally fire to my Arabic channel.

Acceptance Criteria: - [] Admin can define multiple output targets (Vizrt renderers) and label them (Channel 1, Channel 2, etc.) - [] Each graphic type can be assigned to one or more output targets at staging time - [] Operator's queue shows only graphics assigned to their designated output(s) - [] Take-to-air from Channel 1's queue fires only to Channel 1's Vizrt renderer — routing is enforced at API level - [] Control Room dashboard shows the current on-air state per output

Dependencies: C10, C11, C13

C13 — Operator Role Assignment

Sprint: S3 · **Owner:** Both (Tech: assignment backend · Product: assignment UI)

Description:

Assigns Operators to specific output screens at the start of a broadcast. An Operator can be assigned to one or more screens. If an Operator needs to hand off mid-broadcast, the Admin can reassign the screen without the outgoing Operator's action. All assignment changes are logged. An unassigned output is flagged in the Control Room dashboard.

User Story:

As an Admin preparing for an election night broadcast, I want to assign each Operator to specific output screens before broadcast so that responsibilities are clear and cannot be confused during the broadcast.

Acceptance Criteria: - [] Admin assigns Operators to output screens before or during a broadcast - [] An Operator can only take-to-air on their assigned outputs - [] Admin can reassign an output to a different Operator at any time (effective immediately) - [] Unassigned outputs are highlighted in the Control Room dashboard as a warning - [] All assignment changes are logged in C21 with the Admin's name and timestamp

Dependencies: C11, C12, C20

C14 — Real-Time Status Monitor

Sprint: S3 · **Owner:** Product Engineer

Description:

A live status panel showing: what is currently on air (per output), what is next in each queue, queue depth per module, Vizrt connection health, feed connection health, and system performance indicators. Updated in real time via WebSocket push — the Operator never needs to refresh. Visual indicators (green/amber/red) give instant health status at a glance.

User Story:

As an Operator, I want to see at a glance what is on air right now, what is queued next, and whether all systems are healthy so that I am always in control of the broadcast state.

Acceptance Criteria: - [] On-air status updates within 1 second of a take-to-air action - [] Queue depth per module updates in real time - [] Vizrt connection status shows green (connected), amber (degraded), red (disconnected) - [] Feed connection status shows same colour system per connected data feed - [] Status indicators are colour-blind accessible (not colour-only — shape or label also distinguishes state)

Dependencies: C11, C10

C15 — Emergency Override

Sprint: S4 · **Owner:** Both (Tech: override backend · Product: override UI)

Description:

Allows a Senior Editor to immediately pull a graphic from air or clear the entire take-to-air queue for a specific output. Intended for situations where an incorrect graphic has been taken to air or a breaking event requires all scheduled graphics to be discarded. The override action is one click (no

confirmation dialog to avoid delay). Every override is logged immediately with the Senior Editor's identity, the overridden graphic, and a mandatory reason entered after the fact.

User Story:

As a Senior Editor, I need to be able to pull an incorrect graphic from air instantly so that on-air errors are corrected in seconds, not minutes.

Acceptance Criteria: - [] Emergency override button is visible and accessible from the Control Room dashboard without navigation - [] Override executes in $\leq 500\text{ms}$ — the graphic is removed from air - [] Override fires a "black" or "clear" command to Vizrt (removes graphic, does not crash renderer) - [] Override creates an immediate audit log entry (C6) before any reason is entered - [] After the override, the Senior Editor is prompted to enter a mandatory post-hoc reason within 5 minutes — the platform reminds them until entered - [] Override does not require a confirmation dialog (speed is critical)

Dependencies: C10, C16, C6, C20

C16 — Broadcast Event Log

Sprint: S4 · **Owner:** Both (Tech: log API · Product: log UI)

Description:

A chronological log of every on-air event during a broadcast: what went to air, when, on which output, taken by which Operator, approved by which Editor. Different from C6 (which logs all platform actions) — the Broadcast Event Log is the broadcast-specific record, formatted for post-broadcast review. Exportable to PDF for client sign-off and archive purposes.

User Story:

As a Broadcast Manager, I want a clean chronological record of every graphic that went to air during the broadcast so that I can review the production and provide a sign-off report to the client.

Acceptance Criteria: - [] Log displays: timestamp, graphic type, content summary, Operator, Editor who approved, output channel - [] Log is filtered by date/broadcast event (a single election night is one event) - [] Log is exportable to PDF and CSV - [] Log entries are created in real time — visible in the UI immediately after each take-to-air - [] Log is read-only; no user can edit or delete entries

Dependencies: C3, C6

Section 6 — Reson8 Core: AI Content Generation (Sprint 4)

C17 — Auto-Generated Headline

Sprint: S4 · **Owner:** Tech Engineer

Description:

When a significant event is detected (election result posted, goal scored, market threshold crossed), the Sentra Engine automatically generates a broadcast headline in both Arabic and English. The headline is character-limit aware (respects the maximum character count for the target Vizrt scene). The generated headline enters the Producer's queue as a draft — the Producer can edit it, and the Editor must approve it before it reaches air (C7 enforces this).

User Story:

As a Producer, I want a draft headline automatically generated the moment a result is posted so that I have a starting point to work from rather than writing from scratch under time pressure.

Acceptance Criteria: - [] Headline generated in Arabic and English within the ≤4 second SLA from event detection - [] Generated headline respects the character limit of the target Vizrt scene (configurable per scene) - [] Headline enters the Producer queue with status “Draft — Requires Review” (not staged for editor automatically) - [] Producer can accept as-is, edit, or discard the generated headline - [] C7 (Editorial Review Gate) flags the headline as AI-generated and blocks it from air without Editor approval

Dependencies: E5, C1, C7

C18 — Lower-Third Caption Generation

Sprint: S4 · **Owner:** Tech Engineer

Description:

Generates the lower-third text for each graphic type: the short label that appears below the main graphic element (e.g., “LIVE RESULTS — PRESIDENTIAL ELECTION” or “FULL TIME”). Caption generation is template-aware — each graphic type has a caption template that the Engine populates with event-specific data. Output is bilingual (Arabic + English). Character limits are enforced per template.

User Story:

As a Producer, I want lower-third captions to be generated automatically for each graphic so that I do not need to type caption text for every graphic during a fast-moving broadcast.

Acceptance Criteria: - [] Caption generated for each graphic type within the ≤4 second SLA - [] Each graphic type has a defined caption template configured by Admin - [] Caption output is in Arabic and English simultaneously - [] Character limit is respected — if the generated caption exceeds the limit, it is truncated with an alert (not silently cut) - [] Producer can edit the generated caption before staging; edited captions are still flagged as AI-assisted

Dependencies: E5, C8, C17

C19 — Bilingual Broadcast Text

Sprint: S4 · **Owner:** Tech Engineer

Description:

All platform-generated text (headlines, captions, ticker copy) is produced in Arabic and English simultaneously — not translated from one to the other. The Engine generates both language versions from the underlying data, ensuring the Arabic copy is natural broadcast Arabic (not a literal translation of the English). RTL layout is applied automatically to all Arabic text fields. Both language versions are available to the Editor for review simultaneously.

User Story:

As an Arabic-language broadcaster, I need broadcast-quality Arabic copy that reads naturally — not a translation of the English — so that my Arabic output sounds like professional broadcast Arabic.

Acceptance Criteria: - [] Arabic and English versions are generated simultaneously from the same source data (not one translated from the other) - [] Editor sees both language versions side-by-side in the approval screen - [] Arabic text is RTL-formatted in all preview and output surfaces - [] Arabic copy uses broadcast-standard Arabic vocabulary (Modern Standard Arabic unless dialect is specifically configured) - [] Both language versions are sent to Vizrt simultaneously; the broadcaster's Vizrt scene determines which language to display

Dependencies: E1, E5, E9

Section 7 — Reson8 Core: Performance & Reliability (Sprint 4)

C25 — ≤4 Second Wire-to-Air SLA

Sprint: S4 · **Owner:** Tech Engineer

Description:

The platform's defining performance commitment: from the moment a data feed event is ingested to the moment the corresponding graphic is on air via Vizrt, elapsed time must be ≤4 seconds. This includes: feed ingestion, data processing, Sentra Engine enrichment, editorial queue entry, editorial approval, take-to-air, and Vizrt render trigger. The SLA is measured per event and logged. Any breach is flagged in the monitoring dashboard.

User Story:

As a Broadcaster, I need the platform to guarantee that results and events appear on screen within 4 seconds of the official data feed so that my broadcast is never scooped by a competitor reading the same feed.

Acceptance Criteria: - [] End-to-end latency (feed event → on-air graphic) is measured and logged for every event - [] 95th percentile latency under normal broadcast load is ≤4 seconds - [] SLA breaches (>4 seconds) are flagged in real time in the Control Room status monitor - [] Load test at 100 simultaneous events does not cause latency to exceed 4 seconds at the 95th percentile - [] The editorial approval step is not included in the 4-second SLA clock (Editor approval is a human action; the SLA covers the automated portion of the pipeline)

Dependencies: E6, C3, C9, C10

C26 — 99.99% Uptime Target

Sprint: S4 · **Owner:** Tech Engineer

Description:

The platform architecture is designed for zero-failure broadcast events. 99.99% uptime equates to less than 53 minutes of downtime per year. Achieved through: redundant service instances, health monitoring with automatic restart, database replication, and graceful degradation (if a non-critical component fails, the core editorial chain and Vizrt output continue operating). Uptime is measured and reported per tenant.

User Story:

As a Broadcaster, I need the platform to remain operational throughout a live broadcast event so that a system failure never causes a blank screen during live coverage.

Acceptance Criteria: - [] All critical services (editorial chain, Vizrt integration, feed processor) run as minimum 2 instances with automatic failover - [] Health monitoring detects a failed instance within 30 seconds and triggers automatic restart - [] Database replication ensures no data loss on primary database failure - [] Non-critical component failure (notification system, analytics) does not affect editorial chain or Vizrt output - [] Uptime is logged and reportable per tenant per month

Dependencies: C27, C28

C27 — Primary Feed Redundancy

Sprint: S4 · **Owner:** Tech Engineer

Description:

Every data feed has a primary source and at least one configured fallback. If the primary feed becomes unavailable (connection timeout, authentication failure, data gap >30 seconds), the platform automatically switches to the fallback feed without operator intervention. The switch is logged. The primary feed is monitored for restoration and the platform switches back when it recovers.

User Story:

As a Broadcaster covering an election, I need the results data to keep flowing even if the primary official results feed goes down so that I am never left with stale data on air.

Acceptance Criteria: - [] Each configured feed has at minimum one fallback source defined - [] Automatic failover triggers if primary feed has no new data for >30 seconds or connection drops - [] Failover completes within 10 seconds (seamless from editorial perspective) - [] Failover event is logged in C21 and generates a notification to Admin and Operators - [] Primary feed restoration is detected automatically and the platform switches back without manual action - [] Editorial queue continues to function during failover with fallback data

Dependencies: C23

C28 — Broadcast Output Failover

Sprint: S4 · **Owner:** Tech Engineer

Description:

If the primary Vizrt output path becomes unavailable (Viz Engine disconnect, network failure), the platform automatically activates the secondary output path within the SLA. The secondary Vizrt renderer takes over rendering without operator intervention. The broadcast does not go to black. Failover is logged and an alert is sent immediately.

User Story:

As a Broadcaster, I need the graphics output to continue even if the primary Vizrt renderer fails so that on-air graphics never go to black due to a technical fault.

Acceptance Criteria: - ☐ Secondary Vizrt renderer is always in a “warm” standby state (connected, mirroring primary) - ☐ Primary failure is detected within 5 seconds - ☐ Automatic failover to secondary completes within 10 seconds - ☐ The on-air graphic that was live on the primary is reproduced on the secondary within SLA - ☐ Failover event creates an immediate alert to Operators and Admin - ☐ Manual override to switch back to primary is available once it recovers

Dependencies: C10, C23

C29 — On-Prem or Cloud Deployment

Sprint: S4 · **Owner:** Tech Engineer

Description:

The platform deploys in the broadcaster’s own data centre (on-premises) or in a cloud environment, depending on client requirement. On-premises deployment requires no outbound internet dependency for core broadcast operations (the editorial chain, Vizrt integration, and data processing all run locally). Cloud deployment is available as a managed option. Deployment mode is configured at installation time and documented per client.

User Story:

As a Broadcaster with a strict data sovereignty requirement, I need the platform to run entirely within my own infrastructure so that no editorial data leaves my premises.

Acceptance Criteria: - ☐ Platform can be installed and operated on-premises without any internet-dependent components in the critical broadcast path - ☐ Cloud deployment operates with equivalent functionality to on-premises - ☐ Deployment documentation covers both modes - ☐ Data

feeds (election results API, Opta, etc.) are the only outbound internet connections required in on-premises mode - [] Sentra Engine can be deployed on-premises or called via API in cloud mode (configurable)

Dependencies: C22, C24

Section 8 — ORA: System Setup (Sprint 5)

O1 — Election Configuration Wizard

Sprint: S5 · **Owner:** Both (Tech: config backend · Product: wizard UI)

Description:

A step-by-step configuration wizard that an Admin completes before a broadcast to define the election: country, election type (Presidential / Parliamentary / Senate / Local / Referendum), date, total seats available, electoral system rules (first-past-the-post, proportional representation, mixed), round structure (single round or multi-round with advancement rules), and the majority threshold. The configuration drives all downstream modules: results display, seat tally, and map fill logic are all derived from it.

User Story:

As an Admin, I want to configure a new election in a guided wizard so that I do not need to understand the underlying data model — I just answer questions and the system is ready for broadcast.

Acceptance Criteria: - [] Wizard collects: country, election type, date, total seats, electoral system, round structure, majority threshold - [] Each electoral system type loads the correct rules engine (proportional: d'Hondt or largest remainder; FPTP: simple plurality; mixed: configurable) - [] Wizard validates: seat count must be a positive integer; majority threshold must be between 1 and total seats - [] Completed configuration is saveable as a named template for reuse in future elections of the same type - [] Configuration can be edited after creation but before broadcast — changes are logged - [] The wizard can be completed in ≤15 minutes by a non-technical Admin

Dependencies: C20

O2 — Candidate Database

Sprint: S5 · **Owner:** Both (Tech: database backend · Product: management UI)

Description:

The complete registry of all candidates standing in the configured election. Each candidate record contains: full name (Arabic and English), affiliated party, district/constituency, photo, national ID or ballot code, biography, and social media handles. Supports bulk import from CSV (for large candidate sets). Candidate records are linked to constituency data (O4), party data (O3), and live results feed (O5) by candidate ID.

User Story:

As an Admin, I want to import all candidate data before broadcast night so that candidate names, photos, and party affiliations appear correctly on every graphic without manual entry during the broadcast.

Acceptance Criteria: - [] Individual candidate creation form with all required fields: full name (Arabic + English), party, constituency, photo upload, ballot code - [] Bulk import via CSV: Admin uploads a CSV with defined column headers; system validates and imports all valid rows; invalid rows are reported with error reason - [] Photo upload: minimum 400×400px; JPEG or PNG; stored and served at correct resolution for Vizrt profile card - [] Candidate linked to party (O3) and constituency (O4) by ID — unlinked candidates are flagged as warnings - [] Candidate search by name, party, or constituency in management UI - [] Candidate data is available in the results feed binding before broadcast night

Dependencies: O1, O3, O4

O3 — Party Database

Sprint: S5 · **Owner:** Both (Tech: database backend · Product: management UI)

Description:

The registry of all political parties participating in the election. Each party record contains: official full name (Arabic and English), acronym/abbreviation, official colours (primary and secondary, as hex codes), official symbol/logo, leadership names, founding date, and ideological classification. Party colours from this database are used to colour-fill the geographic results map (O16) and colour-code all results graphics automatically.

User Story:

As an Admin, I want to register all parties with their official colours so that every results graphic automatically uses the correct party colours without the Producer or Operator needing to select them.

Acceptance Criteria: - [] Party record fields: full name (Arabic + English), acronym, primary colour (hex), secondary colour (hex), logo upload (SVG or PNG), leadership name - [] Primary colour hex is validated (must be a valid 6-digit hex code) - [] Party logo is stored and served for use in Vizrt profile card graphics - [] Party colours propagate automatically to: results bar fills, map fills (O16), seat tally display - [] Duplicate acronym detection: system warns if two parties have the same acronym - [] Parties without a registered colour use a configurable default colour

Dependencies: O1

O4 — Constituency Mapping

Sprint: S5 · **Owner:** Tech Engineer

Description:

Defines the geographic and administrative structure of the election: every district, constituency, or province configured with its boundaries (for map display), seat allocation, and parent-child hierarchy (National → Province → District). The constituency structure drives the results drill-down (O12), the geographic map (O16), and the seat tally (O10). Geospatial data is loaded from standard GeoJSON format.

User Story:

As an Admin, I want to configure all constituencies with their boundaries and seat allocations before broadcast so that when results arrive, they are automatically placed in the correct geographic location on the map.

Acceptance Criteria: - [] Constituency creation: name (Arabic + English), seat count, parent constituency, geographic boundary (GeoJSON upload) - [] GeoJSON import validated: Admin uploads a GeoJSON file and the system maps each feature to a constituency record - [] Hierarchy enforced: every district must have a province parent; every province must have a national parent - [] Seat counts per constituency must sum to the total seats defined in O1 - [] Constituency data is linkable to results feed events by constituency ID or name matching (configurable mapping) - [] Constituencies without GeoJSON data still work for results display — only the map layer is absent

Dependencies: O1

O5 — Results Feed Integration

Sprint: S5 · **Owner:** Tech Engineer

Description:

Connects to the official election results data feed (typically an API provided by the electoral commission). Configuration includes: API endpoint, authentication method (API key, OAuth, or credentials), field mapping (which API field maps to which platform data field), validation rules (acceptable vote count ranges, % variance detection), and fallback feed configuration. Once configured, the feed is monitored continuously; new data triggers immediate processing through the event pipeline.

User Story:

As a Tech Engineer, I want to configure the results feed connection once so that when results data arrives on election night, it flows automatically into the platform without any manual action.

Acceptance Criteria: - [] Feed connector supports: REST API (JSON/XML), WebSocket, SFTP-delivered files, and direct database connection - [] Authentication supports: API key (header or query param), OAuth 2.0, Basic Auth - [] Field mapping UI: each incoming API field is mapped to a platform field; unmapped fields are surfaced for Admin action - [] Validation rules: Admin can set min/max acceptable values and % change limits per data field; violations trigger O14 (validation layer) - [] Feed health monitoring: last-received timestamp visible; alert if no data received for configurable period (default: 2 minutes) - [] Primary + fallback feed configured (C27) - [] Feed can be tested with a simulated payload before broadcast night

Dependencies: O1, O2, O3, O4, C27

O6 — Seat / Vote Threshold Alerts

Sprint: S5 · **Owner:** Both (Tech: threshold engine · Product: config UI)

Description:

Configurable alert triggers that fire when the results data crosses a defined milestone: a party reaches majority (50%+1 of seats), a candidate crosses a configured seat milestone, a specified percentage of precincts have reported, or concession language is detected in statements. When a threshold is crossed, an alert fires in the editorial queue with the specific threshold that was reached. The alert is a prompt for the editorial team to prepare relevant graphics.

User Story:

As an Editor, I want an automatic alert when a party is about to reach or has reached a majority so that I can prepare the corresponding graphic without watching the numbers manually.

Acceptance Criteria: - [] Admin can configure: majority threshold (default: 50%+1 of total seats), custom seat milestones (e.g., “alert at 100 seats”), % precincts reported milestones (e.g., “alert at 50%, 75%, 100%”) - [] Alert fires within 10 seconds of the threshold being crossed in the data - [] Alert appears in the editorial queue as a dedicated alert item (not embedded in regular queue) - [] Alert contains: threshold that was crossed, current value, party/candidate that triggered it - [] Multiple thresholds can be configured per election; each fires independently - [] Alert is acknowledgeable by Editor (removes it from queue); acknowledgement is logged

Dependencies: O1, O5, C6

O7 — Elections Template Library

Sprint: S5 · **Owner:** Product Engineer

Description:

The set of pre-built Vizrt scenes for the ORA Elections module, all pre-bound to platform data fields via the Vizrt Template Registry (C8).
Templates include: results bar, national map, constituency drill-down map, seat tally display, candidate profile card, party profile card, ticker, seat projection graphic, and breaking news overlay. All templates are bound before broadcast night so no scene configuration is required during the broadcast.

User Story:

As an Operator on election night, I want all graphics to be ready to fire without any configuration so that I am making editorial decisions, not technical ones.

Acceptance Criteria: - [] All listed scene types are loaded into the Vizrt Template Registry with complete data bindings - [] Each binding is tested with a simulated data payload before broadcast night: correct field populates correct scene container - [] Arabic and English text fields are bound to their respective language-specific containers - [] Party colour fills are bound to party colour data from O3 — no manual colour selection required - [] Template health is visible in the pre-broadcast checklist (all bindings green before going live) - [] Admin can add or modify template bindings via the Template Registry UI without code deployment

Dependencies: C8, O3, O4

O8 — Pre-Election Baseline Load

Sprint: S5 · **Owner:** Both (Tech: data ingestion · Product: baseline UI)

Description:

Loads historical election results and pre-election polling data into the platform before broadcast night. Historical results are used for comparative analysis (O15) and the swing visualisation. Polling data provides pre-election context alongside live results. Both datasets are sourced from Admin upload (CSV or API) and are linked to constituency, party, and candidate records.

User Story:

As a Producer preparing election night coverage, I want historical results and current polling data already loaded in the system so that comparative graphics are ready to use without data entry during the broadcast.

Acceptance Criteria: - [] Admin can upload historical results via CSV (structure: constituency, candidate/party, vote count, year) - [] Historical results are linked to current constituency records (O4) and party records (O3) by matching ID or name - [] Polling data upload: supports CSV or API connection to polling agency data feed - [] Baseline load is completable before broadcast night (no live-broadcast dependency) - [] Loaded data is visible in a pre-broadcast data verification screen - [] Mismatched records (historical constituency names that don't match current configuration) are flagged for Admin resolution

Dependencies: O1, O2, O3, O4

Section 9 — ORA: Live Results (Sprint 6)

O9 — Real-Time Vote Count Display

Sprint: S6 · **Owner:** Both (Tech: vote count pipeline · Product: display UI)

Description:

Displays the live vote count for every candidate and party as results are reported by the official feed. Shows both percentage and absolute vote numbers. Updates continuously as new results arrive — each update from the feed triggers a display refresh within the SLA. The display is per-candidate within each constituency and can be aggregated to the national level.

User Story:

As an Operator on election night, I want the vote count graphic to update automatically every time new results come in so that the numbers on screen are always current without me manually updating them.

Acceptance Criteria: - [] Vote count updates within ≤ 4 seconds of new data arriving from the results feed - [] Both percentage (e.g., 47.3%) and absolute vote count (e.g., 2,847,392 votes) are displayed - [] Per-candidate view: all candidates in a constituency ranked by current vote share - [] National aggregate: total votes and percentages across all reported constituencies - [] Partial results clearly labelled (e.g., “47% of precincts reporting”) — not displayed as final - [] Vote counts are formatted with locale-appropriate number separators (Arabic-Indic numerals for Arabic output)

Dependencies: O5, O14

O10 — Seat Tally Display

Sprint: S6 · **Owner:** Both (Tech: seat computation · Product: tally UI)

Description:

Running count of seats won and projected for each party, updated after every declared result. Displays the majority threshold line (derived from O1 configuration) so viewers can see at a glance how close each party is to a governing majority. Distinguishes between declared seats (confirmed results reported) and projected seats (based on partial results, clearly labelled as projection).

User Story:

As a Producer, I want a live seat tally that updates after each result is declared so that I can take a seat tally graphic to air that always reflects the current state of the election.

Acceptance Criteria: - [] Seat count updates within ≤ 4 seconds of each constituency result being declared - [] Majority threshold line displayed and labelled (e.g., “Majority: 151 seats”) - [] Declared seats and projected seats are visually distinct (solid vs. outline bars, or colour coding) - [] Projections are clearly labelled as “Projected” — not displayed as confirmed results - [] Tally covers all registered parties (O3); parties with 0 seats are shown with 0, not omitted

Dependencies: O1, O5, O9, O14

O11 — Lead / Trail Indicators

Sprint: S6 · **Owner:** Both (Tech: ranking logic · Product: indicator UI)

Description:

Live ranking of candidates within each constituency based on current vote share, with visual gain/loss markers comparing the current result against the same constituency's result in the prior election. Automatically identifies the leading candidate and the margin of the lead. Updates continuously as results report. Feeds the map fill logic (O16) — the leading candidate/party colours the constituency on the map.

User Story:

As a Viewer watching election graphics, I want to immediately see who is leading in each area and whether that is a gain or loss compared to last time so that I understand the story of the election without deep knowledge of the history.

Acceptance Criteria: - [] Leading candidate clearly identified per constituency (largest current vote share) - [] Margin of lead displayed (e.g., "Leading by 12.4%") - [] Gain indicator shown when a party leads a constituency they did not hold in the prior election - [] Loss indicator shown when a party trails in a constituency they held in the prior election - [] Indicators update within ≤ 4 seconds of vote count changes - [] Prior election data sourced from O8 (baseline load); indicators are hidden if prior data is not loaded

Dependencies: O5, O8, O9, O14

O12 — Results by Constituency

Sprint: S6 · **Owner:** Both (Tech: drill-down data model · Product: drill-down UI)

Description:

Allows the Producer, Editor, and Operator to drill into results at any geographic level: National → Province → District. Each level shows the results for that specific area, sortable by vote share, margin, or swing. When the Producer stages a constituency-level result graphic, the Vizrt scene is populated with that constituency's data automatically. Drill-down navigation uses breadcrumb UI.

User Story:

As a Producer preparing a constituency-specific result graphic, I want to navigate to the specific constituency and stage the result for that area so that the graphic shows accurate, locally-relevant data.

Acceptance Criteria: - [] Three-level drill-down: National → Province → District, following the hierarchy configured in O4 - [] Each level displays: candidate names, vote counts, percentages, and seat status for that area - [] Results sortable by: current vote share (default), margin of lead, alphabetical by candidate/party - [] When a constituency is selected for a graphic, the scene binding automatically populates with that constituency's data - [] Breadcrumb navigation shows current position in hierarchy; click-through navigates up - [] Search by constituency name returns matching result immediately

Dependencies: O4, O5, O9, O14

O13 — Progressive Results Update

Sprint: S6 · **Owner:** Both (Tech: progress calculation · Product: progress UI)

Description:

Shows the percentage of precincts/polling stations that have reported results, giving viewers and the editorial team a sense of how complete the picture is. Displays a confidence indicator per result: low confidence (< 25% reporting), medium (25–75%), and high (> 75%). The percentage reporting figure appears on relevant graphics so viewers understand they are seeing a partial picture.

User Story:

As an Editor reviewing a results graphic for approval, I want to see what percentage of precincts have reported so that I can decide whether to display the result as live or add a “partial results” caveat.

Acceptance Criteria: - [] % precincts reporting calculated from results feed data and displayed in real time - [] Three confidence tiers: Low (<25% reporting), Medium (25–75%), High (>75%), each with distinct visual treatment - [] Confidence indicator appears on all results graphics (not just a dashboard stat) - [] When 100% of precincts have reported, result is marked as “Final” automatically - [] % reporting updates within ≤4 seconds of new precinct data arriving

Dependencies: O5, O9

O14 — Results Validation Layer

Sprint: S6 · **Owner:** Tech Engineer

Description:

Before any results data enters the editorial queue, the validation layer checks it for statistical anomalies: vote counts that decrease (impossible in most electoral systems), percentage jumps that exceed configured tolerances (e.g., a constituency result that changes by >30% in a single update), or data that conflicts with known totals. Anomalous data is held in a “Flagged” state — it does not proceed to the editorial queue until an Editor explicitly releases it.

User Story:

As an Editor, I need suspect data to be automatically held before it reaches my queue so that I am never put in a position of unknowingly taking an erroneous result to air.

Acceptance Criteria: - [] Validation checks run on every results data update before it enters the processing pipeline - [] Flagging triggers: vote count decrease, % change > configured tolerance (default: 20% per update), total votes > registered voters for constituency - [] Flagged data appears in a separate “Flagged Items” view in the editorial queue, not in the main queue - [] Editor must explicitly release a flagged item to allow it to proceed - [] Flagged item release is logged in C6 with the Editor’s name and timestamp - [] Non-anomalous data is not delayed by the validation layer (validation adds ≤200ms to processing)

Dependencies: O5, C6

O15 — Comparative Results

Sprint: S6 · **Owner:** Both (Tech: comparison logic · Product: comparison UI)

Description:

Side-by-side display of the current election result against the prior election result for the same constituency. Shows: vote share change (+ or – percentage points), absolute vote change, and seat change. The comparison uses the historical data loaded in O8. Graphics can be staged showing the comparative view — the Vizrt scene shows both the current result and the historical benchmark simultaneously.

User Story:

As a Producer, I want to show viewers how each party’s result compares to the last election so that I can tell the story of where votes have moved, not just what the current count is.

Acceptance Criteria: - [] Comparative data available for every constituency that has historical data in O8 - [] Comparison displays: current share, prior share, change (+ or – percentage points), direction indicator (gained/lost) - [] Constituencies without historical data display current results only (no

comparison); this state is clearly labelled - [] Comparative Vizrt scene shows both current and prior data simultaneously in the configured layout - [] Comparison updates in real time as current results update (prior data is static)

Dependencies: O8, O9, O11

Section 10 — ORA: Interactive Map (Sprint 7)

O16 — Geographic Results Map

Sprint: S7 · **Owner:** Both (Tech: map data service · Product: map UI)

Description:

A national geographic map where each constituency is colour-filled by the currently leading party or candidate. Colour fills use the official party colours from O3 automatically. The map updates in real time as results arrive — a constituency that switches from one party's colour to another updates within the SLA. Constituencies with no result yet are displayed in a neutral colour with a “No Result” label. The map is the signature visual of the election broadcast.

User Story:

As a Viewer watching the election broadcast, I want to see a geographic map that shows me at a glance which party is leading across the country so that I can understand the geographic pattern of the result.

Acceptance Criteria: - [] Map renders all configured constituencies (O4) using their GeoJSON boundaries - [] Each constituency fills with the leading party's primary colour (O3) automatically - [] Fill updates within ≤4 seconds of a new leading party being determined by O11 - [] Constituencies with no result display in a neutral grey with a “Pending” state - [] Map renders correctly in both the preview pane and the Vizrt output - [] The national map is the default view; drill-down is available (O17)

Dependencies: O3, O4, O11

O17 — Regional Drill-Down

Sprint: S7 · **Owner:** Both (Tech: geospatial hierarchy · Product: drill-down UI)

Description:

From the national map (O16), the Producer or Operator can drill into a specific region to see a zoomed map of that region's constituencies. Navigation follows the hierarchy defined in O4 (National → Province → District). Each level shows the same colour-fill logic as the national map, but for the sub-constituencies within the selected area. Breadcrumb navigation shows the current position and allows navigation back to higher levels.

User Story:

As an Operator covering a closely contested province, I want to zoom into that province's map to show viewers the constituency-level breakdown of that region's results.

Acceptance Criteria: - [] Click on a province on the national map zooms to the province-level map showing that province's districts - [] Click on a district zooms to the district level (lowest configured hierarchy level) - [] Breadcrumb navigation shows: National > [Province Name] > [District Name]; clicking any level navigates up - [] At each level, the colour-fill logic applies for that level's leading parties - [] A "back to National" button is always visible - [] Drill-down navigation is available in both the Producer/Editor UI and the Vizrt output scene

Dependencies: O4, O16

O18 — Colour-Coded Party Performance

Sprint: S7 · **Owner:** Tech Engineer

Description:

Automatically applies each party's configured official colours (from O3) to all results displays — not just the map. Party colour is used consistently across: results bars, seat tally, lead/trail indicators, and candidate profile cards. The colour system is automatic — no Producer or Operator needs to select a colour. A party's colour is set once in O3 and propagates to every graphic that references that party.

User Story:

As a Producer, I want party colours to appear automatically on every graphic without me selecting them manually so that visual consistency is guaranteed across the entire broadcast.

Acceptance Criteria: - [] Party primary colour (O3) is applied automatically to all graphics referencing that party: results bars, seat tally rows, map fills, profile card borders - [] Colour propagation is immediate — updating a party's colour in O3 updates all graphics referencing that party within 30 seconds - [] Parties with no configured colour receive a system-assigned

default colour (from a pre-configured palette of distinct colours) - [] Colour contrast check: system warns Admin if a party's configured colour has insufficient contrast against the template background (WCAG AA compliance)

Dependencies: O3, O16

Section 11 — ORA: Election Preparations (Sprint 7)

O23 — Historical Election Data

Sprint: S7 · **Owner:** Tech Engineer

Description:

Historical results from prior elections, pre-loaded via O8, are accessible in the live results views for comparison. The historical dataset covers: prior vote counts, seat allocations, and winning parties per constituency. This data powers the comparative results display (O15) and the lead/trail gain/loss indicators (O11). Historical data is static — it does not update during the broadcast.

User Story:

As a Producer, I want historical election results to be pre-loaded and accessible during the broadcast so that I can stage comparative graphics without any reference look-up.

Acceptance Criteria: - [] Historical data loaded via O8 is available in all live result views as a comparison layer - [] Historical data is clearly labelled with the election year it represents - [] Multiple prior elections can be stored; the most recent prior election is the default comparison - [] Historical data does not interfere with live results processing — it is a read-only reference layer - [] Admin can update or replace historical data without affecting live results

Dependencies: O8, O15

O24 — Polling Data Integration

Sprint: S7 · **Owner:** Both (Tech: polling feed connector · Product: polling display UI)

Description:

Pre-election polling data, loaded before broadcast night, provides a pre-results benchmark. When results begin arriving, the polling data remains visible as a contextual reference — showing how the actual results compare to what was predicted. Polling data is displayed alongside live results, clearly labelled as “Pre-Election Polling” to avoid confusion with official results.

User Story:

As an Editor, I want to see polling data next to live results so that I can identify significant departures from expectations and prepare graphics that explain the difference.

Acceptance Criteria: - [] Polling data loaded via O8 is displayed in a dedicated “Polling” column/row in the results view - [] Polling data is labelled “Pre-Election Polling — [date/source]” throughout - [] Polling data never appears in a graphic alongside live results without a clear label distinguishing it - [] Polling data is per-party at the national level (constituency-level polling data is optional) - [] Polling data remains static during the broadcast; only Admin can update it

Dependencies: O8

O25 — Graphics Template Pre-Load

Sprint: S7 · **Owner:** Product Engineer

Description:

A pre-broadcast verification step confirming all ORA Vizrt templates (O7) are bound, connected, and tested before broadcast begins. Generates a pre-broadcast checklist showing the status of each template binding. Any binding with an error is flagged for resolution. When all bindings are confirmed healthy, the template library is “locked” for broadcast — no binding changes during a live broadcast without explicit Admin override.

User Story:

As a Tech Engineer on the day of the broadcast, I want a pre-broadcast template verification checklist so that I can confirm all graphics are ready before going live, not discover issues mid-broadcast.

Acceptance Criteria: - [] Pre-broadcast checklist shows: all template bindings and their status (bound/unbound/error) - [] Each binding can be individually tested with a sample data payload from the verification screen - [] All bindings must be green before the “Ready for Broadcast” status is set - [] Admin can lock the template configuration before broadcast — changes

require a lock override - [] Lock override is logged in C21 with the Admin's identity and timestamp - [] Checklist is exportable as a PDF sign-off document

Dependencies: O7, C8

Section 12 — ORA: Breaking News (Sprint 7)

O30 — Elections Breaking Feed

Sprint: S7 · **Owner:** Tech Engineer

Description:

A dedicated breaking news stream filtered specifically to elections-related content: concession statements, victory declarations, electoral commission announcements, candidate statements, protest reports, and disputed results. Uses the Sentra Engine's breaking detection (E7) scoped to election topics. Breaking items surface in the ORA editorial queue with a distinct "Breaking" label, separately from the results queue, so editors can act on them without interrupting results processing.

User Story:

As an Editor covering election night, I want a filtered breaking news feed showing only elections-related breaking items so that I am not distracted by unrelated breaking news while managing results graphics.

Acceptance Criteria: - [] Breaking feed displays only content classified as Breaking or Major Breaking AND related to the configured election (country, parties, candidates) - [] Election-specific breaking items appear in a dedicated "Breaking" section of the ORA editorial queue - [] Each item includes: severity label, source, detection timestamp, summary, credibility score - [] Items are sorted by severity (Major Breaking first) then recency - [] Non-election breaking items are excluded from the ORA breaking feed (they appear in FLASH, not ORA) - [] Filter can be widened by Admin (e.g., include neighbouring country coverage) via configuration

Dependencies: E7, E12, C20

O31 — Results Threshold Alert

Sprint: S7 · **Owner:** Tech Engineer

Description:

Fires an alert in the editorial queue when a results milestone configured in O6 is crossed. Distinct from the general breaking feed (O30) — this alert is triggered by the official results data feed, not social media or news wires. The alert signals that a pre-defined electoral event has occurred (majority reached, seat milestone, 100% of votes counted) and prompts the editorial team to take the corresponding graphic to air.

User Story:

As an Operator, I want an automatic alert when a party crosses a majority so that I can immediately take the majority announcement graphic to air without watching the seat tally constantly.

Acceptance Criteria: - [] Alert fires within 10 seconds of the configured threshold being crossed in the official results data - [] Alert appears at the top of the editorial queue with a “Threshold Reached” status and the specific threshold description (e.g., “Party X has reached majority: 151 seats”) - [] Alert is linked to a pre-staged graphic of the appropriate type (e.g., majority announcement graphic), allowing one-click staging - [] Alert is distinct from the breaking news feed (O30) and appears in its own alert section - [] All fired alerts are logged in C6 with the crossed threshold value and timestamp

Dependencies: O6, O5, C6

Feature Count Summary — Phase 1 PRD

Section	Feature IDs	Count
Sentra Engine	E1–E12	12
Core — Editorial Chain	C1–C7	7
Core — Vizrt Integration	C8–C10	3
Core — Access & Security	C20–C24	5
Core — Control Room	C11–C16	6
Core — AI Content	C17–C19	3
Core — Performance	C25–C29	5
ORA — System Setup	O1–O8	8
ORA — Live Results	O9–O15	7
ORA — Interactive Map	O16–O18	3
ORA — Election Preparations	O23–O25	3
ORA — Breaking News	O30–O31	2

Section	Feature IDs	Count
Total Phase 1 PRD		64

Stretch features (O19–O22 Entity Profiles, O26–O29 Social Sentiment) are not included in this PRD. They will be added as a Sprint 8 addendum if capacity allows.

*ID8 Media · info@id8media.com · Qatar: +974 6626 4422
Reson8 Platform — Phase 1 PRD · June 2026*